

First-Class Refinement Types for Scala 3

Matt Bovel

Viktor Kunčák

Martin Odersky

EPFL

We present a new design and prototype implementation of refinement types for Scala, integrated directly in the compiler. We prove type soundness of a core, pure calculus.

A *refinement type* refines a base type T with a predicate $p(x)$, written as $\{x: T \text{ with } p(x)\}$. It denotes the subset of values $x: T$ for which $p(x)$ holds.

For example $\{x: \text{Int with } x > 0\}$ is the type of positive integers and $\{x: \text{List[Int]} \text{ with } x.\text{nonEmpty}\}$ the type of non-empty integer lists.

First-class. We extend the Scala 3 type checker itself so that refinement types are ordinary types, and integrate with other features such as subtyping and implicit resolution:

```
given Ordering[Int] = ...
def max[T: Ordering, U <: T](xs: List[U]): U = ...
type Even = {v: Int with v % 2 == 0}
max(List(2, 4, 6)): Even
```

or with overload resolution:

```
def min(l: List[Int] with l.isSorted): Int = ...
def min(l: List[Int]): Int = ...
val l: List[Int] with l.isSorted = ...
min(l) // calls the first overload
```

Example. Safe methods on length-indexed vectors:

```
def concat(l: Vec, r: Vec):
  {v: Vec with size(v) == size(l) + size(r)} = ...
def sum(l: Vec, r: Vec with size(l) == size(r)):
  {v: Vec with size(v) == size(l)} = ...
```

Refinement types ensure that the size of a concatenation is the sum of the input sizes:

```
val v3: {v: Vec with size(v) == 3} = ...
val v4: {v: Vec with size(v) == 4} = ...
concat(v3, v4): {v: Vec with size(v) == 7}
```

Solver. A type $\{x: T \text{ with } p(x)\}$ is a subtype of $\{y: S \text{ with } q(y)\}$ if $T <: S$ and $p(x)$ implies $q(x)$ for all x .

To check implications, we use a custom solver combining equality reasoning via *E-Graphs*, term normalization, inlining and constant-folding.

Inference. Refinement types are not inferred by default. This avoids changing how Scala normally types expressions and prevents an explosion of fine-grained types.

Instead, when a refinement type is expected, the compiler attempts to *selfify* the expression: it gives $e: T$ the type $\{x: T \text{ with } x == e\}$ when possible. This allows terms to be lifted into types when they are valid predicates:

```
val y: (Int with y == n + 2) = n + 2
```

Runtime checks. When a value's properties cannot be verified statically, runtime checks can be performed using pattern matching:

```
val idRegex = "[a-zA-Z_][a-zA-Z0-9_]*$"
type ID = {s: String with s.matches(idRegex)}
"my-id" match
  case id: ID => // id matches idRegex
  case _      => // does not match idRegex
```

Mechanization. We formalize a core calculus with refinement types using a fuel-bounded *definitional interpreter* and *semantic types*, denoting types as sets of values, in the Rocq Prover.

Our calculus extends System $F_{\lambda\mu}$ with dependent function types, dependent pairs, sum types, union and intersection types, refinement types, loops, and positive equi-recursive types.

Our semantics ensures *partial correctness*: e has type $\{x: T \text{ with } p(x)\}$ if, whenever e terminates with a value v , the predicate $p(v)$ holds. This is a key difference from prior systems, which are designed around termination.



matt.bovel.net/scala-refinement-types

